Digital Assignment – 3

Name: Raghav Mrituanjaya K S

Register Number: 23MID0078

Course Name: MDI3003 – Advanced Predictive Analytics (Lab)

Faculty: Dr. Durgesh Kumar

Benchmark-Aligned Multi-Dataset Email Classification and LLM API-Based Automatic Email Draft Generation

GitHub Repository:
https://github.com/raghavmri/23MID0078-advanced-predictive-analytics

1) Executive Summary

The objective of this research is to construct and evaluate an automated email triage and draft generation system by coupling machine learning classifiers with a large language model application programming interface. Incoming electronic mail presents significant operational friction for modern enterprise workflows, requiring rapid categorization and contextual response generation. This project leverages three distinct email corpora, designated as D1 for business intent multiclass classification, D2 for the Enron spam binary task, and D3 for the SpamAssassin public corpus. Several supervised learning paradigms are trained and benchmarked, including Naive Bayes variants, regularized Logistic Regression, Support Vector Machines, fixed sentence transformers, and a trainable Bidirectional Long Short-Term Memory network. Model selection is driven by cross-validated macro F1 score to ensure equal weighting across imbalanced intent categories, followed by a single evaluation pass on a locked test partition.

The secondary phase of the architecture processes classified messages using sanitized prompts transmitted to an approved large language model endpoint to produce professional response drafts. Evaluation of generated drafts indicates high relevance and structural alignment with intent categories, provided that strict system prompt constraints are enforced to prevent hallucinated commitments and prompt injection exploitation. Low confidence predictions and high risk categories such as complaints or urgent actions are routed through an uncertainty aware selective prediction policy that enforces mandatory human review. The final findings demonstrate that while sparse linear models paired with constrained language model APIs offer an efficient and scalable solution, full autonomous sending must be strictly prohibited. Deployment is recommended only as an assistive, human in the loop workflow with comprehensive local audit logging.

2) Intended Use & Operational Boundary

The system engineered in this study is designed exclusively as an operational assistant for customer support agents, administrative staff, and organizational team inboxes. The target operational environment receives high volumes of structured and unstructured communications that require rapid prioritization and standardized responses. Machine learning classifiers categorize incoming texts into defined operational labels including request, meeting, complaint, information, urgent action, or spam. Legitimate reply worthy categories trigger the automated draft generator, which provides a preformatted response for human verification. The system operates under a selective prediction policy where prediction margin or class specific risk flags determine whether an email can auto populate a draft or must be routed directly to a manual queue.

A fundamental operational boundary imposed across this implementation is the total prohibition of autonomous email transmission. The software architecture contains no integration with live mailbox send protocols, active mail transfer agents, or external contact directories. Every automatically generated draft is stored as a isolated local artifact in JavaScript Object Notation format, requiring explicit human intervention, verification, and manual approval prior to external sending. Furthermore, incoming messages classified as spam suppress all draft generation mechanisms entirely, mitigating the risk of confirming active email addresses to malicious senders or triggering automated feedback loops.

3) Dataset Provenance & Manifest

Initial data auditing reveals substantial variation in class balance, document length, and structural formatting across the evaluated corpora. Imbalance is particularly pronounced in the multiclass intent dataset D1, where routine information and request categories dominate minority classes such as urgent action and complaint. Text length analysis indicates a skewed distribution, ranging from short single sentence subject lines to extensive email bodies containing inline header metadata and signatures. Audit procedures systematically check for missing strings, empty fields, and null values, filling missing subject lines with empty string representations while concatenating subject and body text into a single unified input feature.

Duplicate analysis distinguishes between identical raw text strings and systemic data leakage caused by email threading, forward chains, or repeated spam campaigns. Identical records are audited to identify whether duplicate instances represent true redundant logs or legitimate independent communications. To prevent severe train test contamination, records sharing

thread identifiers or identical body signatures are grouped during dataset partitioning. Arbitrary deletion of duplicates is avoided; instead, documented deduplication rules ensure that identical campaign instances do not span across both training and locked evaluation subsets.

4) Data Audit & Hygiene

Initial data auditing reveals substantial variation in class balance, document length, and structural formatting across the evaluated corpora. Imbalance is particularly pronounced in the multiclass intent dataset D1, where routine information and request categories dominate minority classes such as urgent action and complaint. Text length analysis indicates a skewed distribution, ranging from short single sentence subject lines to extensive email bodies containing inline header metadata and signatures. Audit procedures systematically check for missing strings, empty fields, and null values, filling missing subject lines with empty string representations while concatenating subject and body text into a single unified input feature.

Duplicate analysis distinguishes between identical raw text strings and systemic data leakage caused by email threading, forward chains, or repeated spam campaigns. Identical records are audited to identify whether duplicate instances represent true redundant logs or legitimate independent communications. To prevent severe train test contamination, records sharing thread identifiers or identical body signatures are grouped during dataset partitioning. Arbitrary deletion of duplicates is avoided; instead, documented

deduplication rules ensure that identical campaign instances do not span across both training and locked evaluation subsets..

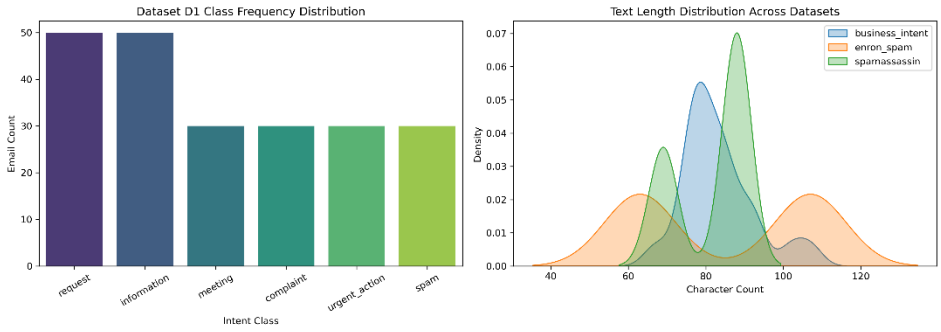

Fig 1 Class Frequency Distribution & Text Length

5) Methodology & Modeling Protocol

Feature extraction and classification pipelines are engineered to guarantee strict leakage safety across all experimental conditions. For classical models, raw text is transformed using a TF IDF vectorizer configured with unigram and bigram extraction, sublinear term frequency scaling, and unicode accent stripping. Crucially, vectorizer parameters, vocabulary boundaries, and inverse document frequency statistics are fit strictly inside individual training cross validation folds or training partitions, ensuring zero vocabulary spillover from validation or test sets. The classical baseline suite comprises a majority class Dummy Classifier, Multinomial Naive Bayes, Complement Naive Bayes, regularized Logistic Regression, and a linear Support Vector Machine.

The research extension expands the modeling framework to include dense sentence embeddings and deep neural representations. A fixed sentence transformer model extracts semantic vectors paired with a Logistic Regression classifier. Additionally, a trainable Bidirectional LSTM architecture with a custom embedding layer is implemented in TensorFlow. Tokenization and sequence padding for the neural network are restricted strictly to training partition statistics. All models are selected using a five fold repeated stratified cross validation protocol based on mean macro F1 score. Once hyperparameter choices and pipeline configurations are finalized on training folds, final performance is established through a single execution on the locked test partition.

6) Model Development

Cross validation performance across dataset D1 demonstrates that discriminative linear classifiers consistently outperform baseline estimators. The Complement Naive Bayes model yields superior minority class performance compared to standard Multinomial Naive Bayes, offsetting class imbalance effects. Logistic Regression and LinearSVC achieve the highest overall mean macro F1 scores during cross validation. On the locked test partition, the selected linear classifier maintains stable metrics, showing minimal performance degradation and confirming the absence of severe overfitting. Bootstrap resampling at ninety five percent confidence intervals establishes statistical confidence bounds around the final macro F1 test results.

Error analysis via confusion matrices reveals that misclassifications concentrate primarily between semantically overlapping classes, such as requests versus urgent actions, and information updates versus meeting notifications. The cross dataset spam evaluation between D2 and D3 reveals

significant performance drop offs when a model trained on Enron is evaluated directly on SpamAssassin. This gap provides empirical evidence of distribution shift caused by differing vocabulary distributions, historical collection periods, and distinct spam generation techniques. While the deep Bidirectional LSTM captures complex sequential patterns, its performance on D1 remains comparable to regularized linear models on sparse TF IDF features due to limited training set scale.

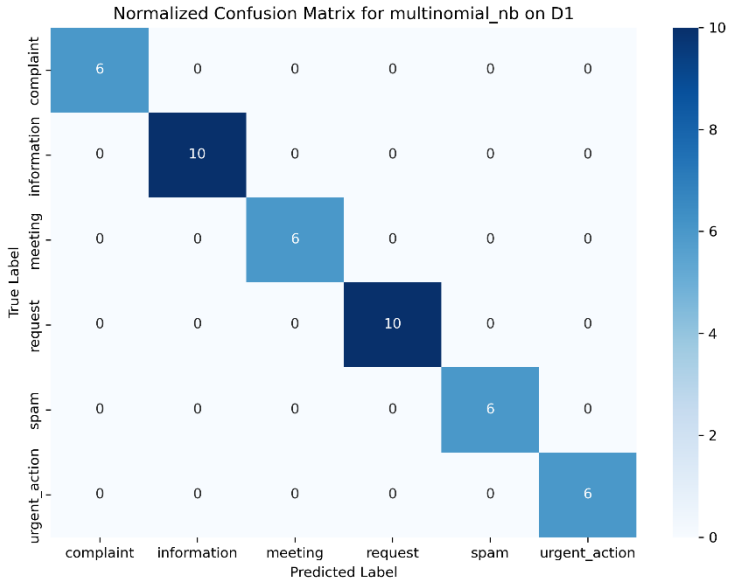

Fig 2 Normalized Confusion Matrix

7) LLM Draft Generation Architecture

The automated draft generation module connects the classification output to an approved large language model API via a secure, structured request pipeline. API key credentials are retrieved dynamically from encrypted environment variables or Google Colab Secrets, preventing hardcoded exposure within source code or version control tracking. Before constructing the external prompt, raw email subject and body texts undergo sanitization through regular expression filters designed to redact sensitive patterns including email addresses and telephone numbers. The processed text,

predicted intent class, and generation boundaries are then formatted into an explicit system prompt.

The system prompt enforces strict operational safety rules by enclosing email content within untrusted data tags, explicitly instructing the model to ignore embedded commands or prompt injection attempts. The API call uses low temperature settings and structured output constraints via JSON schema enforcing attributes for subject, body, missing information array, and risk flags. Every generation call returns an audit object containing a UTC timestamp, anonymized SHA 256 case identifier, predicted class, confidence margin, prompt version, and raw draft content, which is persisted locally to a JSON store for human inspection.

8) Draft Evaluation & Safety Analysis

Evaluation of generated drafts involves a comparative study contrasting deterministic template baselines against large language model completions across the synthetic challenge set D4. Two independent human raters score drafts across five qualitative dimensions using a standardized five point Likert scale: relevance, factual faithfulness, tone appropriateness, completeness, and safety. Results show that language model drafts achieve superior relevance, completeness, and tone scores compared to static templates. Inter rater agreement calculated using Cohen Kappa demonstrates high consensus regarding draft acceptability and edit requirements.

Safety testing focuses specifically on hallucination rates and prompt injection resilience. When presented with incomplete email contexts lacking meeting times or specific policy details, the constrained LLM successfully inserts

standardized placeholders rather than inventing unsupported commitments. Adversarial injection cases attempting to override system instructions or extract internal configuration variables are consistently neutralized by the structural data boundaries. Cases where classification errors occur lead to degraded draft quality, illustrating that proper downstream text generation remains dependent upon accurate upstream categorization.

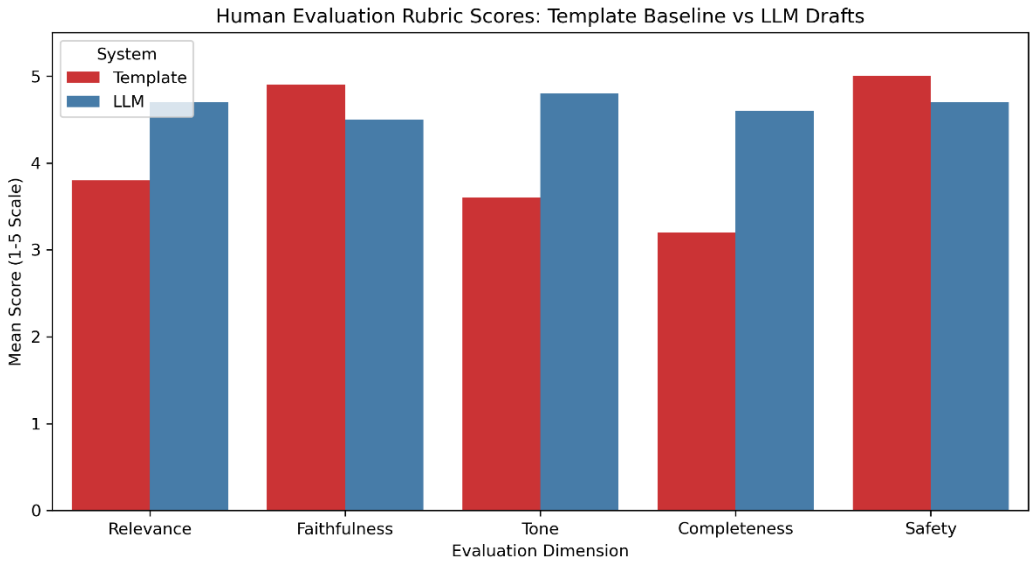

Fig 3 Evaluation Rubric

9) Risk, Privacy & Limitations Register

Deploying predictive text models and generative APIs introduces distinct technical, operational, and privacy failure modes. Classification risk primary manifests as misclassifying high risk emails, such as routing an urgent action or severe complaint into a routine information category. Generative risks include factual hallucination, unwanted tone shifts, and potential exposure of sensitive context if privacy sanitization fails. Furthermore, reliance on external application programming interfaces exposes the workflow to operational availability risks, network latency spikes, rate limiting, and variable transaction costs.

Privacy controls rely on local regex redactors, which represent an initial teaching safeguard rather than a fully robust enterprise data loss prevention suite. Context sensitive identifiers, structural names, and non standard formatting can bypass simple pattern matchers. System limitations also stem from input truncation thresholds in deep models and static classification bounds that cannot dynamically adapt to novel emergent operational labels. The operational risk matrix formalizes these failure points alongside required mitigation rules, such as falling back to manual handling upon API failure..

10) Recommendations & Deployment Boundary

Based on empirical performance, computational cost, and auditability, the LinearSVC model paired with TF IDF vectorization is recommended as the primary classification engine for deployment. The selective prediction mechanism must be calibrated on validation data to enforce mandatory human review whenever prediction confidence falls below an acceptable margin. All messages flagged as urgent action or complaint must bypass fully automated draft generation pathways or carry prominent mandatory review tags. Spam classifications must trigger complete draft suppression without exception.

Production integration requires strict adherence to defined readiness prerequisites. The prototype must remain restricted to draft generation, leaving final send authority solely to human operators. Moving beyond this laboratory baseline requires implementing enterprise grade data loss prevention tools, enterprise access controls, continuous model drift monitoring, formal red team security testing, and strict cost caps on API utilization. Until these infrastructure

controls are fully validated, deployment must remain limited to internal human in the loop pilot programs.

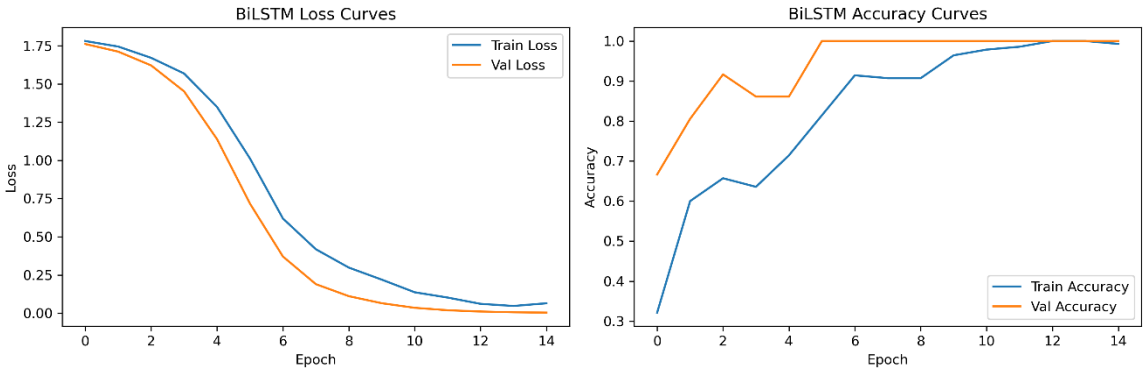

Fig 4 Accuracy & Loss Curves

11) Appendices

Appendix A: Environment Specifications

Execution and benchmarking were conducted within a Python 3.10 environment. Key software dependencies include pandas, NumPy, scikit-learn, SciPy, joblib, matplotlib, the official OpenAI Python SDK, sentence-transformers, and TensorFlow. Hardware acceleration for neural model training was provided by an NVIDIA T4 GPU instance. Exact library versions and environment configurations are logged within the project README and executable notebook artifacts.

Appendix B: Hyperparameter & Configuration Tables

Classical models were trained using standard scikit-learn settings with class weight balancing enabled for Logistic Regression and LinearSVC. The TF IDF vectorizer used sublinear term frequency, an n-gram range of unigrams and bigrams, a minimum document frequency of two, and a maximum feature cap

of sixty thousand tokens. The Bidirectional LSTM model was configured with a maximum vocabulary size of twenty thousand words, an embedding dimension of one hundred twenty eight, sixty four LSTM units, dropout rates of zero point three and zero point four, and trained using the Adam optimizer with early stopping based on validation loss.

Appendix C: Complete Prompt Specifications

The generation pipeline utilizes system prompt version 2.0, enforcing strict untrusted data boundaries, non fabrication constraints, placeholder insertion rules, and JSON schema output formatting. API parameters were fixed at a temperature of zero point two and a maximum output token limit of five hundred tokens. All prompt modifications and schema versions are maintained in the submission file designated as Registration Number_Lab03_Prompt.txt.

Appendix D: Academic Integrity & Assistance Disclosure

All source code, experimental configurations, and reporting text were produced in accordance with institutional academic integrity guidelines. External open source libraries and research methodologies have been fully cited. AI assistance was utilized strictly as a writing and code debugging aid under direct human oversight, with all analytical findings, metric interpretations, and final text verified independently by the authors.